



My program is ok – am I? Computing freshmen's experiences of doing programming assignments

Päivi Kinnunen & Beth Simon

To cite this article: Päivi Kinnunen & Beth Simon (2012) My program is ok – am I? Computing freshmen's experiences of doing programming assignments, Computer Science Education, 22:1, 1-28, DOI: [10.1080/08993408.2012.655091](https://doi.org/10.1080/08993408.2012.655091)

To link to this article: <https://doi.org/10.1080/08993408.2012.655091>



Published online: 14 Mar 2012.



Submit your article to this journal [↗](#)



Article views: 2434



View related articles [↗](#)



Citing articles: 26 View citing articles [↗](#)

My program is ok – am I? Computing freshmen’s experiences of doing programming assignments

Päivi Kinnunen^{a*} and Beth Simon^b

^a*School of Educational Sciences and Psychology, University of Eastern Finland, Finland;*

^b*Computer Science and Engineering Department, University of California, San Diego, USA*

(Received 9 March 2011; final version received 20 December 2011)

This article provides insight into how computing majors experience the process of doing programming assignments in their first programming course. This grounded theory study sheds light on the various processes and contexts through which students constantly assess their self-efficacy as a programmer. The data consists of a series of four interviews conducted with a purposeful sample of nine computer science majors in a research intensive state university in the United States. Use of the constant comparative method elicited two forms of results. First, we identified six stages of doing a programming assignment. Analysis captures the dimensional variation in students’ experiences with programming assignments on a detailed level. We identified a core category resulting from students’ reflected emotions in conjunction with self-efficacy assessment. We provide a descriptive model of how computer science majors build their self-efficacy perceptions, reported via four narratives. Our key findings are that some students reflect negative views of their efficacy, even after having a positive programming experience and that in other situations, students having negative programming experiences still have a positive outlook on their efficacy. We consider these findings in light of possible languages and support structures for introductory programming courses.

Keywords: computing majors; CS1; self-efficacy; experience; programming assignment

1. Introduction

The purpose of this study is to explore the experiences students have with programming assignments in an introductory computing course. Students document that they spend a great deal of time doing required programming assignments in this course – at least as much as all other coursework combined (lecture, reading, etc.). This drove our interest

*Corresponding author. Email: paivi.kinnunen@uef.fi

because of the potential impact on students' views about computing and majoring in computing. From a broader perspective, students' experiences with programming assignments may increase our knowledge of factors relevant to retention of computing students. Poor learning outcomes and low passing rates are well-known problems in introductory programming courses (CS1) in many universities (see Lister et al. 2004). Therefore, in order to be able to plan effective pedagogical interventions, one long-term objective of this study is to shed light on the processes students go through as they study programming.

Researchers in the field of computing education research have put a great deal of effort into studies of students' success and experienced difficulties in CS1 courses (often, to help to explain retention/attrition phenomena). We identify four different kinds of studies:

- Studies that focus on the content of the course; which content students have difficulties learning (Goldman et al., 2008; Schulte & Bennedsen, 2006) or the kind of misconceptions students have of core concepts (Sorva, 2008).
- Studies that look at student-related factors such as students' previous academic success (Cantwell Wilson, 2002), cognitive skills (Robins, Rountree, & Rountree, 2003), psychological facets (Wiedenbeck, 2005), or learning skills (Simon et al., 2006) in relation to success.
- Studies that focus on the factors related to the learning environment, such as, social/cultural aspects of the classroom (e.g. Garvin-Doxas & Barker, 2004).
- Studies that explore the applicability of pedagogical approaches such as context-based computing (media computation, games, and robotics) (Tew, Fowler & Guzdial, 2005).

Much existing computer science education (CSE)-related literature focuses on cognitive or behavioral aspects of the learning process. These are primarily *etic* accounts – description from the point of view of an observer. *Emic* accounts – detailed studies of students' experiences of the learning process – have attracted less interest.¹ Examples of studies using emic accounts are Berglund and Eckerdal (2006) who study students' motives for taking the class, and Sorva (2008) and Eckerdahl and Thuné (2005) who report students' perceptions of constructs and concepts. This study also takes an emic viewpoint of introductory students' experiences with programming assignments. By exploring such a key (and time-consuming) part of the CS1 experience, we seek to gain understanding of the possible impact on students' views about computing and majoring in computing. Considered in light of existing research in this area, we hope to provide insight into why certain approaches or techniques seem to

work well, and focus adopters' attention on the aspects of their curriculum, which may foster positive (or alleviate negative) experiences.

This article is one of the four papers we have written on this topic. Kinnunen and Simon (2010b) discusses the first part of our results, Kinnunen and Simon (2010a) describes our methodology in more detail manner, and Kinnunen and Simon (2011) looks at some of our results in the light of social cognitive theory. The emphasis of the current article is on the second part of our results (results from the selective coding).

In this article we first describe the empirical part of the study and after that draw the connections to the existing literature and theories. We justify this choice by our research approach, grounded theory, whose basic idea is to generate a theory/model rather than verifying one (Glaser & Strauss 1967/1999).

2. Research questions and methodology

Our general goal was to explore how computer science majors experience the process of doing programming assignments in a CS1 course. This specific course was taught with a media computation approach and using pair programming for assignments. Given the lack of emic accounts in this area, we designed a grounded theory study where the phenomenon is the *perception* of the experience reported by students shortly after a programming assignment was due. It should be noted that we describe students' perception of programming assignments, not their actual experience. Perceptions are of interest because they may impact students' views of themselves as successful majors.

Our data collection consisted of repeated interviews throughout students' first programming course in fall 2009. This format supported the grounded theory approach by which the subject of interest can be adjusted based on the observed experience. Repeated contact also led to an increase in students' comfort with relating more personal thoughts.

2.1. Sample and the media computation-based CS1 course

At a university orientation meeting, a purposeful sample of 18 freshmen computing majors at a large, selective research institution in the US were invited to be interviewed about their computing history, thoughts about their major, and expectations concerning computing classes. Women and minority students were over-represented in the sample to support study of a wide variety of experiences. We selected nine students for the study based on those who were comfortable speaking and maintained diversity in the data (e.g. diversity in gender, ethnicity, and prior computing experience). Of those nine interviewees six were males, three females. Hispanic, Asian and Caucasian ethnicities were evenly represented in the

data (three students each). Four of the interviewees did not have prior programming experience. The rest of the students had some experience (e.g. had taken a high-school computer science class or had been a part of a high-school robotics team), but no one had significant background in programming. At the beginning of the course, they varied in their confidence about keeping their computing major. Some students were very confident that they would graduate with a computing degree while others had many interests and were planning to “give CS a go” to see if it would be something they would enjoy. Our interviewee sampling criterion sought to represent a large variety of experiences to promote development of a model applicable to a great range of computing learners, not to support quantitative representation of experiences.

The course was for 10 weeks including weekly 50 min supporting closed labs, nine programming assignments (done using pair programming), weekly quizzes, one mid-term, and one final. The course had around 150 students. Weekly programming assignments were designed to engage the students deeply with the key concepts and constructs. Students used standard Java to do the assignments. An example assignment (week 4) is “Create a simple collage with three copies of an image, all with at least one filter applied (one of your own design). Modify a picture to flip horizontally and vertically a portion of the picture.” For more detailed description of the course, see Simon, Kinnunen, Zaskis, and Porter (2010).

2.2. *Data collection, analysis and results*

Face to face interviews with the nine students lasting 20–55 min were conducted five times during the 10-week term in weeks 3, 5, 7, 8 and 10. We report on the more substantive interviews from weeks 5, 7, 8, and 10 (week 3 served as a warm-up). An initial semi-structured interview protocol asking students to recall and reflect on the programming assignment just completed (usually within the past one to three days) was continually refined and modified until week 7 using the constant comparison method (see Kinnunen & Simon 2010a, 2010b). The last interview plan deviated by focusing more on the students’ overall course experience. An identical protocol was used with each interviewee in a particular week.

Our protocol covered topics including encountered difficulties and episodes leading to affective reactions (positive or negative) during the process of doing programming assignments. The same themes were always discussed with each of the interviewees. Through this refinement of the protocol, we have more trust in the theory or proto-theory outlined, because it is less directed by the preconceptions of the researchers and more directed by the actual perceptions of the students.

For instance, we added questions concerning students' emotional reactions towards the process of doing the programming assignments to our protocol. Emotional reactions seemed to be a theme that the interviewees kept on mentioning during the first interviews without the interviewer prompting it. At the data analysis phase, emotions proved to play an essential role in our evolving theory of how students experience the process of doing programming assignments and assess their programming related self-efficacy.

According to Strauss and Corbin (1990, p. 251), "The purpose of grounded theory is to specify the conditions that give rise to specific sets of action/interaction pertaining to a phenomenon and the resulting consequences." Here, we seek to specify and describe in a deep, detailed way the phenomenon of students' reflective experience with doing a programming assignment via standard Grounded Theory analysis (including open, axial and selective coding) as described by Strauss and Corbin (1990). For a thorough account of this process, see the papers by Kinnunen and Simon (2010a, 2010b). The resulting theory generated from selective coding in a grounded theory study may be reported as a series of narratives expressing the range and depth of variation observed in the study of the phenomenon. We identify four variations on students' programming experiences in conjunction with their assessment of self-efficacy. Self-efficacy perception is defined here as "beliefs in one's capabilities to organize and execute the courses of action required to produce given attainments" (Bandura, 1997, p. 3).

2.3. *Validity discussion*

We consider the quality of this work from three viewpoints: measures taken before data collection, during data collection and analysis, and validity of the results. In our discussion, we focus on trustworthiness of the research process, the richness and scope of the data, workability and evocativeness of the results.

We sought to support the internal validity – or trustworthiness (Lincoln & Guba, 1985) – of the research project both before and during the data collection phase. We used prolonged engagement and peer debriefing as suggested by Lincoln and Guba (1985) to assist in correct understanding of interviewees' statements. Both researchers were previously students in a CS1 course and one researcher regularly teaches the course. In addition, both authors have previously studied CS1 students and thus gained some knowledge of the challenges students encounter. Involving the course instructor in the analysis poses a threat to validity. We took measures to reduce the impact of this threat: the instructor was not involved in recruiting students or conducting interviews. The other (interviewing) author was not involved with

organizing or delivering the course. Interview data was anonymized before analysis and no changes were made to the delivery of the course as a response to any preliminary results.

During analysis, which overlapped with data collection, results emerged from debriefing discussions between the two authors, thus reducing the possibility that one researcher saw only what she wanted to see in the data. Each preliminary category had to be supported by the data and make sense to both researchers. Additionally, theoretical sensitivity (e.g. being aware of the “subtleties of meaning of data” (Strauss & Corbin 1990, p. 41)) is an important quality the researcher should bring to bear in any grounded theory study. Therefore, having the instructor, who knows the context and the content of the course well, involved with the analysis lends strength to analysis.

Cohen, Manion, and Morrison (2000) suggest that the validity of a qualitative study can be assessed through the richness and scope of the data that is achieved and the extent of triangulation. Our research setting allowed relatively lot of control over the data collection phase. We were able to adjust the interview protocol to explore emerging themes and repeated interviews supported a trusting relationship where many interviewees felt comfortable sharing their thoughts freely. Interviewees’ increased level of comfort during the interviews became evident during the face to face meetings. For instance, interviewees’ body language became more relaxed over the time and they shared more personal information with the interviewer on their own initiative. On the other hand, lack of triangulation data may mean that we have missed some aspects of students’ experiences. For instance, direct observation would seem important for study of actual programming processes, but logistical considerations precluded this. Students often study at times difficult (and likely intrusive) to observe. Lincoln and Guba (1985) also suggest that one aspect of trustworthiness is the degree to which the results are transferable. We have described features of our interviewees and the course context to provide the reader adequate information to judge whether current results could be relevant in another place and time.

We can also discuss the validity of the results using the concepts of workability and evocativeness (Heikkinen, Huttunen, & Syrjälä, 2007). Workability refers to functional and productive results that enable readers to create workable practices and provoke fruitful discussion on the subject. In the discussion section, we explore how our results support the adoption of specific pedagogical practices and the implication for further curricular developments. Evocativeness refers to study’s ability to provoke the reader to think about things in a different or new way. We hope that the detailed description of students’ experiences (especially the four narratives described in section 3.2.2) highlights aspects of students’ studying process that instructors do not normally get to see.

3. Results

Analysis of students' stories revealed a pervasive, often intense phenomenon of self-efficacy assessment; computing majors were constantly assessing their self-efficacy as they did programming assignments. Their self-efficacy assessment was often externalized through emotional reactions to some of the short-term phases during the process of doing the assignment.

From axial coding of students' emotional experiences, a six-phase process (Figure 1) emerged defining the programming assignment experience. Each of these stages included emotion-induced mini-phenomena that described students' experiences during that phase. We have discussed the *Getting started*, *Encountering difficulties*, and *Dealing with difficulties* phases in our previous publication (Kinnunen & Simon 2010b). Therefore, we will only summarize them in this article (Table 1).

Next, in section 3.1 we report on the axial coding results from the remaining three stages of the programming process experience (*Succeeding*, *Submitting*, and *Stopping*). We discuss how students experienced the remaining stages as they did the assignments. Finally, in section 3.2 we elaborate on the core category of self-efficacy assessment that we came up with during the selective coding. In section 3.2.2 we use the results of the axial coding as our starting point and identify four varying ways how students use their experiences of the process of doing the assignments to assess their programming related self-efficacy.

3.1. Results of axial coding: stages

In this section, we will shortly report on the six phenomena (Success not due to own actions, Success due to own actions, It works!, I will be done, Really finished, I am done) that took place during the *Succeeding*, *Submitting*, and *Stopping* phases. At axial coding phase, we aimed at abstracting and compressing the data. For instance, we looked for different ways students experienced the various situations as they did the assignments.

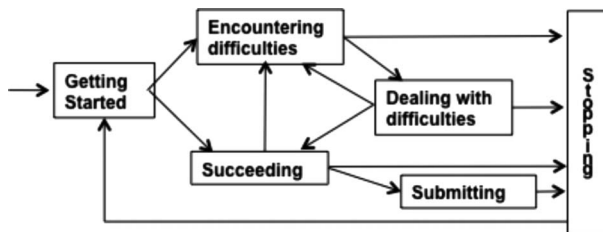


Figure 1. Stages of experiencing programming assignments.

Table 1. Summary of the stages.

Stage	Phenomena	Description	Emotional consequence
<i>Getting started</i>	It's Greek to me	Being completely lost regarding the programming assignment specifications. Meant that the process of doing the assignment was stymied right at the outset.	Confusion and puzzlement
	Ok, what now?	Characterized by students' perception of their ability or knowledge to do the assignment.	Puzzled, confused, and "froze up"
<i>Encountering difficulties</i>	Hit by lightning	Experience comes completely out of the blue, occurs in the context of a state of student confidence about their program ("I'm sure it's right"), and leaves them with little idea about what is wrong or what to do next.	Sense of being taken back and almost betrayed by the error, dazed, puzzled, confused, frustrated, overwhelmed, and annoyed
	Rapid change	In an elaboration of the struck by lightning experience, students were affected by the rapidity of emotional change that encountering an error caused.	Shock
<i>Dealing with difficulties</i>	Hamster wheel	Does not use feedback to guide following actions.	Despair
	Feedback guided	Uses varying depth of information in guiding actions (binary, low and high levels of feedback identified).	Puzzled, confused
<i>Succeeding</i>	Success not due to own actions	Success occurs because of luck or partner's skills.	Relief
	Success due to own actions	Avoid a difficulty: success as being able to avoid a problem even if it means using unauthorized means; Rely on self: actively contributing to the event where a success was achieved. The episodes leading to this experience often relate to the process of solving a problem.	Relief, happy, proud, accomplished, stupid, pathetic
	It works!	Students' experiences related to getting the code working on the first try.	Happy, accomplished, excited

(continued)

Table 1. (Continued).

Stage	Phenomena	Description	Emotional consequence
<i>Submitting</i>	I will be done	Getting some concrete confirmation that one is on the right track and that finishing the assignment is possible in the near future.	Happy, excited, gratified, and proud
	Really finished	Immediate experiences right after submitting the assignment is the final sign that the assignment is really finished and there is nothing else to be done.	Happy, proud, accomplished, gratified, satisfied, excited, and relaxed
<i>Stopping</i>	I am done	Reflects general aspects and the consequence of being done.	Happy, relief

We find reporting on the axial coding results important for two reasons: first, the axial coding results were the basis for the next level analysis. By reporting them we want to increase the transparency of our analysis process. Second, as instructors we found it informative to learn about the range of students' experiences at various stages. The following six phenomena together with the other six we reported in Kinnunen and Simon (2010b) provide readers a possibility to peek into students' experiences and reflections that are usually not accessible to instructors.

3.1.1. *Succeeding*

Success not due to own actions: A sense of great relief was experienced by students who were able to get their code to compile or run even when they lacked a deep level of understanding of their code. Their experience of success resembles the actions of someone playing the lottery. The students hit the compile button (or run the code) and hope for the best; but, they really have no idea what might happen. When they "win", students feel relief. Another manifestation of "Success not due to own actions" relates to situations, where success was due to partner. This is an experience where the student takes no active role in the critical problem solving moment, but gets to enjoy the end result. Sometimes students did work with their partner initially, and put in some work in dealing with the difficulty. But when the team did not find a solution, the student and the partner would take a break during which the partner had a revelation leading to the solution. In these cases, the student still experiences enjoyment, but the student also expresses explicit knowledge of the fact that he did not contribute to the solution

at all and perhaps that, even after the fact, he does not understand why it has worked.

Success due to own actions: When students knew what was wrong with the code but did not know how to fix it, they sometimes used unauthorized means to avoid the difficulty or get around it (avoiding the difficulty). For example, a student used photo-editing software to change the color of their input picture, thus avoiding difficulties concerning color variations (which simplified the assignment). The student described the situation saying that she and her partner had just “dodged a bullet” since they found a way to avoid dealing with a problematic part of the assignment. Another aspect of the same “Success due to own actions” phenomenon is the episode where students were able to be successful using authorized means (relying on self). This experience complements the “Success due to partner” experience in that here students themselves were actively contributing to the event where a success was achieved. The episodes leading to this experience often related to the process of solving a problem with a partner although it also included episodes where a student was solely responsible for actions leading to success. Being able to find a solution to a difficulty based on their own actions left students often feeling relief, happy, accomplished and proud but sometimes also stupid and pathetic. Negative feelings related to severe hindsight experiences. Additionally, this experience of being able to solve a difficulty also contributed to the student’s meta-cognitive notions about their own knowledge level.

It works! Getting a piece of code to compile and run is a clear indication for students that they are succeeding. These experiences related to getting the code working on the first try and left students feeling happy, excited, and accomplished. It also gave a boost to students’ self-efficacy assessment.

I will be done: Getting some concrete confirmation that one is on the right track and that finishing the assignment is possible in the near future left students feeling good and proud that they had got that far. When students got part of the assignment working, it reassured them that they were on the right track, even if some tasks remained. The completion of the whole programming assignment meant they were now able to relax and not worry about doing another assignment for a while. During this moment, students were glad that they got through the programming assignment and had done a good job. This experience left students feeling accomplished, glad, gratified, proud and happy.

3.1.2. Submitting

After getting the programming assignment done, students submitted it (electronically). Even though submitting itself happens in a very short

time period, it provoked experiences that related specifically to submitting and not, for example, succeeding.

Really finished: Submitting the assignment is the final sign that the assignment is really finished and there is nothing else to be done. It also meant that “week’s worth of work” was now done. This conveys the feeling of contentment for a job well done – or at least happiness that they did not need to spend any more time on it. The experience of being done left students feeling happy, proud, accomplished, gratified, satisfied, excited, and relaxed.

3.1.3. Stopping

The stopping phase refers either to finishing an assignment or taking a temporary break from doing it. However, most of the emotional reactions related to the occasions when students had finished the whole assignment and reflected either their immediate experiences of being finished or the entire process they had gone through.

I’m done: Finishing the long process of doing the assignment resulted in several distinctive experiences. On the one hand, students were happy to be done after spending a lot of time doing the assignment. The long hours spent doing the assignment led to happiness in “just being done”. On the other hand, being finished also meant that now students were able to stop worrying about the assignment and focus on other things; they were relieved to be able to spend more time on other studies or other aspects of the course. Finally, being done with the assignment was also a sign for students that they had understood the topic and that they now were able to direct their focus to something else. Getting confirmation that they had learned and understood the content/topic led to relief.

3.2. Results of selective coding: programming ability evaluation

The phenomena in the previous section capture a range of individual experiences and emerged from our process of open and axial coding. Through the process of developing and reviewing the results of those coding phases, one theme seemed to be both prominent and also quite important to students-emotionally triggered self-efficacy assessments. When students reflected on emotionally triggered experiences with programming, their stories were often permeated with explicit or implicit references as to how their feelings about the experience impacted their potential for success in the course, their personal level of capability in programming, and their concerns about whether they were as capable as their peers. During Selective coding, we selected students’ emotionally triggered self-efficacy assessments as our core category, and re-analyzed our data from the viewpoint of self-efficacy assessments.

3.2.1. Core category: emotionally triggered programming ability evaluation

Programming-Induced Self-Efficacy Assessment is the phrase we will use to describe the experiences students related as viewed through the lens of self-efficacy. The vast majority of the emotionally laden episodes reflected students' beliefs about their ability to do the assignment (or students' programming ability more generally). We found a range of explicit and implicit self-efficacy statements in the data. The great number of explicit self-efficacy statements (e.g. "... *it was like we had no idea which part would make it work and we were just trying to wrap our heads around [it]*" [I4-J]²) led us to choose emotionally triggered experiences in conjunction with self-efficacy assessment as our core category. Selective coding, through the lens of self-efficacy, revealed less explicit self-efficacy statements, which became an important part of the analysis.

I know after PSA 5, I was very tired, but this PSA, we were walking out and we were like, "That was fun. Let's do that again". ... Yeah, we had a lot of fun, 'cause our picture was completely ridiculous, and when you have a picture like that, you can't help but have a little fun. [I3-D].

Students' experiences with self-assessment varied. In addition to clearly positive and negative assessments, we also found few episodes where students had conflicting thoughts about how they were doing and what was their ability to solve the problem (waffling). This often happened in the situation where students expected to be able to solve the problem but for some reason the solution was not working or it took several attempts to get the code working. These episodes shed light to the possible transition phase between positive and negative self-efficacy assessment and thus provide information on how self-efficacy assessments form.

We got a lot of "out of bounds" errors. And trying to figure out how exactly we were getting those – we knew generally how we were getting them, but we were trying to figure out how much, and so that was kinda confusing. And we thought we had all the right numbers, and then it's be wrong and we'd be puzzled and look at it and try some other stuff and eventually got it working. [I3-D]

Programming-Induced Self-Efficacy Assessment was influenced by four factors: the stage in the programming process, students' geo/programo-location (i.e. their beliefs about where they are and where they should go next in the programming assignment process), students' expectations, and external forces. In Table 2, these factors are shown in the structure of the grounded theory paradigm model to clarify how they contribute to the story of how students construct their self-efficacy beliefs.

Table 2. Factors Influencing Programming-Induced Self-Efficacy Assessments.

Causal condition	A student is in a CS1 course and does the programming assignments as a part of the course
Phenomenon	Emotionally triggered <i>experiences</i> (e.g. hit by lightning) in conjunction with <i>self-efficacy assessment</i>
Context	A stage, geo/programo-location
Intervening conditions	Expectations, external forces
Action/interaction strategies	Stage-specific (mostly found in dealing with difficulties phase)
Consequence	Emotions

Next we will look at the some of the factors influencing self-efficacy assessment in more detail.

Context: The context in which self-efficacy assessments occurred was impacted by two factors: the stage in the process (see Figure 1) and the geo/programo-location of the student.

Stage: Stages refer to different phases that students go through during the process of doing the assignment (getting started, encountering difficulty ...). The process stage in which self-efficacy assessment often took place had the expected impact on the result of that evaluation-the earlier in the programming assignment process you are evaluating yourself, the more you see yourself as in trouble or not performing to standards. Evaluations in the later stages of succeeding, submitting, and stopping tend to be more positive-reflecting some success. However, the mere fact of being in a finishing process stage did not ensure a positive self-evaluation. Even after students were done with an assignment, they could find ways to talk negatively about themselves. These included concerns that the partner had really done all the work, that others in the class had finished with much more ease (or in less time), and that, in hindsight, the student believed that he should have been capable of doing better.

Geo/programo-location: Geo/programo-location refers to students' sense of where they were located (cognitively) in the programming process and what next steps they should take. These experiences influenced their self-evaluations. In general, these experiences fell into two broad states: *completely lost* and *on some track*. When students would find themselves completely lost, they could either be completely stuck and unable to make any progress on their own, hoping for time, a partner or a TA to come to the rescue. Other times, despite being completely lost, they would take action, but that action would be completely blind-not informed in any way by the current situation. Additionally, the results of those actions were not utilized by students to provide more information on where they were or the value of that particular path. Instead actions

were taken with the hope that a random move would get them past their current problem.

In contrast, when students were on some track, they would be using some information to inform their actions-but their confidence or evidence that this was a correct track varied from very little confidence to certainty that things were going well. In the first case, a great deal of concern about one's abilities was often experienced-some of it as negative as expressed by those who were completely lost. Students often were quite hard on themselves at these times, which could be distracting and stressful. Even when students were on track, their evaluation of themselves could be positive or negative-depending on where they placed the responsibility for that success-on their own actions and understanding or on the work of others (TAs and partners).

Intervening conditions: The data revealed two different kinds of intervening conditions that affected students' self-efficacy assessment. On the one hand, there were the *expectations* students had regarding self, others, and the process of doing the assignment. On the other hand, there were *external forces*, such as programming partners, that impacted how students perceived the situation.

(a) Expectations

Students experience with self-efficacy (positive or negative) was strongly influenced by their expectations. Students brought expectations regarding a broad range of things:

- (1) The fluency with the process of doing the programming assignment (time, the amount of help needed): Students used fluency with the process as a cue in assessing their abilities. For example, students compared the time spent on an assignment with previous experiences. Spending less time indicated that they were competent and getting better at programming. Students similarly considered how much help they needed from other students or TAs.
- (2) The student's performance compared to others (time, outcome, fluency of the process): Students used other students' performance as a comparison point. Students constructed their perceptions of ability by comparing their own outcomes (edited pictures, sounds) and the perceived easiness of doing the assignment with their perceptions of their peers in the course. For instance, in some cases students reported feeling bad because other students managed to finish their assignment faster while also producing a nicer outcome.
- (3) The student's perception of their own abilities (cognitive abilities – I understand, I am able to help others): Expectations relating to

one's own cognitive abilities came up in episodes where students reflected on their understanding of the topic/assignment. Positive reflections included delighted notions that "this assignment sounds easy" or "this is similar to what we've done before" (thus implying the student thought he knew how to do it).

In cases where students' expectations were not met, they sometimes experienced rapid shifts in emotions; from positive/neutral to strongly negative. Often, encountering a difficulty led to a very strong, negative emotional reaction that had intense impacts on students' beliefs about themselves – they were "struck down by lightening" and at times betrayed by their expectations. The rapid change from the highs of their expectations to the lows triggered by feedback from the compiler (or running the program) had an additional impact on students-often draining them of energy and causing them to exclaim "oh, no, not again".

(b) External forces

A surprising range of external forces also contributed to students self-evaluations. External forces led to extra concerns regarding self-efficacy by introducing complications probably not intended by the instructor. These complications included issues specific to the course but outside the realm of the intended programming assignment (deadlines, getting input into the right format) and partner issues (concern that partner was dragging one down or that the partner was so much better than oneself, scheduling time with the partner). The self-evaluations occurring as a result of external factors were almost always negative in flavor, with a notable exception occurring when partnerships worked well.

Programming partners played a focal role as an external force that had a potential to have either positive or negative effect on students' self-efficacy. In supporting partner relationships, partners helped each other both cognitively and emotionally-thus reducing stress levels and contributing to a positive experience in general. However, in unsupportive partner relationships the partner's direct negative feedback often directly contributed to negative self-efficacy beliefs.

Time played an important role as an external factor. Approaching deadlines caused students stress. Physiological state (e.g. being tired after working a long time) often resulted in negative emotions and negatively affected students' self-efficacy assessment.

The students' awareness that some difficulties took place in the context of the course lessened negative feelings and negative self-efficacy assessment resulting from those "course-related" difficulties. Since students knew that the instructor "would not give us an impossible assignment" dealing with a difficulty lessened negative self-efficacy assessment.

Table 3. Summary of the four narratives.

	Context		Intervening conditions	
	Stage	Geo/ programo- location	Expectations	External sources
I'm not good at this ("— —" -story)	• getting started • encountering difficulty • dealing with difficulties	off-track	• comparison with self: ◦ fluency of the process ◦ cognitive abilities • comparison with others	• partner • the type of the assignment
Severe hindsight ("+ —" -story)	• encountering difficulty • succeeding • stopping	on-track	• comparison with self: ◦ fluency of the process (time, efficiency) ◦ cognitive abilities	• partner
I'm good at this ("++" -story)	• getting started • succeeding • submitting • stopping	on-track	• comparison with self: ◦ fluency of the process ◦ cognitive abilities • comparison with others	• partner
I will get there eventually ("— +" -story)	• dealing with a difficulty	off-track	• comparison with self	• I see this problem in a context of being in the course

3.2.2. Variations in programming-induced self-evaluation

In this section, we will describe the process of programming-induced self-evaluation in four varying contexts shown in Table 3. The structure of the reporting of these stories is modeled from the grounded theory literature in other disciplines including that modeled in Strauss and Corbin (1990). We would like to stress that the four narratives we are about to present are not individual cases. They are constructions of students' experiences from the corpus of the whole data.

3.2.2.1. Negative Programming Episode → Negative Self-Efficacy Assessment (hereafter referred as "— —" -story. The first "—" sign refers to Negative Programming Episode, the second "—" sign refers to Negative Self-Efficacy Assessment). Many programming episodes that

had a negative outcome (e.g. the program did not compile or the outcome was not what students wanted it to be) left students with a negative self-evaluation of themselves. These episodes tended to happen in context of the getting started and dealing with a difficulty phases. In addition, all negative programming episodes that led to negative self-evaluations related to the context where students were cognitively off-track, that is, students were stuck, lost, and did not know what to do next. For instance, during the getting started phase, not knowing what to do externalized as two different experiences – “It’s Greek to me” and “Ok, what now?”. The former refers to the situation where the student did not understand the written assignment and the latter to the situation where the student does not know how to proceed even though he understands what the assignment is asking him to do.

Students’ expectations played a central role as an intervening condition in their processes of building self-efficacy assessment. Unmet expectations concerning fluency with the process and cognitive abilities led to low self-evaluation. For instance, often students expected the code to be correct and the compiler/run time errors came as a total surprise. In a sense, students were “Hit by lightning” in the discovery of the error.

I had always thought I placed the code correctly, and it gave me like compiling errors out the wazoo and I thought, “Wait, this is not actually – this is not actually what’s supposed to happen. I’m a little lost here.” [I5-J]

At other times students’ perceptions of their own skills were deflated by seeing other students finishing the assignment more quickly and/or with an outstanding outcome. Unmet expectations left students feeling stupid and lost.

I felt stupid I guess you could say because I’ve seen other people be like alright, da, da, da, da, da. They do stuff that’s like way beyond what I could do at my current level, even though we’re in the same class. . . . Like there are people that really get it and it takes them like less than three hours to finish and they create something that looks outstanding and they did something that goes above and beyond what’s called for in the PSA. So it makes you feel a little inferior, like okay. [I2-F]

Students used several strategies when they encountered a difficulty. Many students mentioned that they discussed with their programming partner, asked help from TAs, other students, or friends who were not in the same course. Students also tried to think and try out some ideas to see how the outcome changed or to figure out, which line the error was on. Students also used programming assignments they had done in previous weeks and the textbook to get ideas about solving the current problem. However, sometimes if the strategies were inefficient and students got tired, feedback-oriented strategies changed to trial and error strategies or

doing the same thing over and over again (“Hamster wheel” experience). Additionally, in several cases, students did not mention any strategies to deal with the difficulty. Many of these “no strategy” situations related to the getting started phase. Moreover, even if student used a strategy to deal with the difficulty, it did not guarantee that the strategy was efficient (as perceived by the student).

Like if we got stuck we'd throw questions at one another to help us to get thinking, like to get the gears moving. So we'd ask questions where we were like, well if this is true, then this should happen, right. Well then let's try this. ... So we'd throw questions back at one another to see if we could get something moving to help get our brains start back up. ... It helped a little bit for some things, but for the most part, no ... 'Cause we were both stumped and that's just like pulling things out of the air so it doesn't really help. [I2-F]

There were several intervening conditions, which affected the students' choice of strategy (or lack of it). Factors that kept students from adopting strategies included the type of the assignment (had to start from the scratch → did not know how to do that) and programming partner related issues (e.g. the partner takes charge of the assignment impeding the other student in contributing). In cases where strategies were employed, the choice of the strategy depended on how capable students were in solving the problem. For instance, students first tried to solve the problem by themselves and only if that did not help, would they ask a TA.

There were also factors that limited students' choice of strategy (external forces, course specific). One was the academic integrity rule of the course, which guided students to seek help from their own partner or TA rather than other students or friends.

I'd like to ask them [students who “get it”] for help, but I don't want to – 'cause I'm always worried about that academic integrity policy. Like saying you could only work with your partner on your programming assignments. I'm like well, I really want to ask them because they get this, but I don't want to infringe 'cause I don't want to break the policy because I'm not asking for them to do my code or anything and I'm not asking to see their code, but I don't want to walk the line for that. So I just stay with my partner and the TAs. [I2-F]

However, sometimes exceptions were made. If students were working alone, having to deal with difficulties, and the deadline was approaching, then students sought help from other sources, such as friends, even if that was not in line with the course guidelines. An approaching deadline combined with being stuck resulted in concerns about likely academic success. Other intervening factors related to students' own resources.

Being stressed and tired (dehydrated, “dilapidated”) or having low energy levels contributed to feelings of being disoriented, apprehensive, and frustrated. These cognitive, emotional, and physiological sensations affected students’ self-evaluation assessment.

There is hope even though this narrative portrays a negative scenario, which seems to negatively impact students’ learning experiences. The data included some incidents that illustrated how the students’ interpretation of negative situations changed as they gained more experience in doing assignments. Towards the end of the course, dealing with a programming-related difficulty did not cause as negative a response as it did earlier in the course.

But I mean I felt kind of stupid when I didn’t put X minus minus, but it wasn’t like last time. I didn’t say, “Oh, my god, I’m so stupid.” I was like, “Oh, okay, that’s how you do it.” [I4-B]

3.2.2.2. Positive Programming Episode → Negative Self-Efficacy Assessment (“+ –” -story). Sometimes, even when the programming episode had a positive outcome in the form of a functioning program, students experienced severe hindsight and had a low assessment of their self-efficacy. Most of these episodes unfolded in the context of the succeeding phase although encountering a difficulty and stopping phases were represented in the data, too. Severe hindsight happened in the context where students finally had a working program, but had been struggling. This negative self-efficacy assessment related to two intertwined intervening conditions: students’ expectations relating to their own cognitive abilities and their fluency with the process.

After figuring out their problem, students often reflected on the process of encountering the difficulty and dealing with a difficulty phase and ended up with severe hindsight. Students’ expectations regarding their own cognitive abilities were evident when they were looking back and the problem seemed simple and the answer obvious. These realizations lead students to feel pathetic and stupid. In addition to expectations regarding cognitive ability, students also dealt with expectations regarding their fluency with the programming process (e.g. the time they spent getting it working or fixing a bug). In hindsight, students felt that they should have been able to solve the problem much faster and more efficiently.

The time required solving the problem injected variation into students’ experiences; students’ reflections on the efficiency of the process and emotional consequences were seen in relation to the time students had spent on the task.

I believe that was a lot longer [time] than we should have been [spending on the assignment] because they say you should spend at least six hours in the

lab each week, but combined with what we did we were eight hours long, which seems a little sketchy to me 'cause a lot of people said they finished in three to four hours. . . . As for the time it took, I already discussed that, felt kind of long, like we should have been able to do it more quick I guess you could say; faster and more efficiently. [I2-F]

At other times, the severity of the negative self-assessment was moderated by time spent on dealing with a problem; below the student implies that it was “sort of OK” because he spent only a little time to find the simple mistake that caused the problem.

[student had made a small mistake (such as mixing greater than or less than) while writing the code] I mean, I did feel stupid when I found out that mistake was that. I'm like, “Stupid mistake.” But it's not like I was too hard on myself because I was doing something I was looking for four hours; I didn't spend hours looking for that little mistake. I was like, “Okay, whatever.” [I4-E]

Programming partners provided yet another external force (specific to the course) as an intervening condition. In some cases, the partner's input played a crucial role in coming up with a solution and having a positive outcome in the programming process. However, the effect the partner had on students' self-efficacy assessment varied. In some cases, it was not so bad. The partner just happened to come up with the solution first this time. These incidents did not seem to affect students' self-efficacy assessment. In other cases, in a non-supporting partner relationship, the non-supportive atmosphere between the partners left the student feeling bad and useless. Both the process of working on the assignment (the partner did all the work) and the partner's negative comments or actions (the partner being mad, the partner verbally criticizing the student) left the student with a negative perception of their self-efficacy (“I'm not useful/able to contribute”).

3.2.2.3. Positive Programming Episode → Positive Self-Efficacy Assessment (“++”-story). In many cases positive programming episodes also resulted in positive self-efficacy assessments. These positive assessments most often happened in the context of succeeding, submitting, and stopping. However, sometimes positive episodes together with positive self-assessments occurred also in a context of getting started phase. The common context for all episodes was the students' experience of being on-track, having an idea of what to do next or having a good understanding of the topic at the end of the process.

Cues, which led students to interpret that the process of doing the assignment was going well, related to students' expectations of their fluency with the process, their own cognitive abilities, or physiological

reactions to the process of doing the assignment. Expectations relating to their fluency with the process included the need for help while doing the assignment. The fact that students needed less help than usual from TAs or other students to do the assignment was interpreted as a sign of fluency with the process. In fact, a confirmation of students' own evaluations as to whether or not they were on track were incidents where other students asked them for assistance and they were able to help.

I felt accomplished and happy [because] on top of finishing mine, I helped two other friends and a random person... explaining to [them] helped me understand it even more. Yeah, it just increased the feeling of being proud and happy and accomplished. [I5-A]

Expectations relating to cognitive abilities included students' reflections on how efficiently they were able to get the program working or assignment done. The initial evaluation of whether or not students reported feeling competent doing the assignment occurred when students first read the assignment. Positive expectations (having confidence in one's own abilities) were based on previous experiences with similar assignments or the more general process of doing the assignments.

So I felt happy when I read the PSA assignment because I thought, "Wow. This is actually – sounds a lot easier than the other things that I've had to do before," because the first part I thought was – well, my first impression was, "Okay. Part 1, copy, paste. Done. Next part. This looks like – almost like a collage method almost. Done." And then the next part was – well, I didn't know what to do, but reversing a sound sounded easy. [Laughs] [I4-J]

Other instances of meeting or exceeding expectations concerning one's own cognitive abilities were episodes where something worked on the first try. Students interpreted these as a sign of really understanding the topic. Additionally, the amount of time spent played a central role in students' assessment of their own abilities. Students compared the time spent to their own expectations and to how long other students took to complete the assignment. Students interpreted it as a good sign if it took less time than they thought at the beginning of the process or compared with previous assignments. In addition, students also used the time other students needed for their assignments as a cue to assess their own progress. Finally, sometimes students found a simple solution to a problem they originally thought would be much harder to solve. This experience reflected the students' contentment concerning their ability to avoid a hard problem. Meeting, or even exceeding, one's own expectations concerning their problem solving ability were sometimes described as "dodging a bullet".

Physiological reactions to the process also served as a cue to students. For instance, if students felt fluent in doing the assignment, it caused a

minimum amount of stress. The positive benefit of being on track or understanding was seen to relate to future challenges.

I felt good that I knew what to do in case I ever wanted to do it again. Like now that I'm working on PSA 5, we know a lot of what to do 'cause we had to create a collage again. We're like oh, well that's basically like combining PSA 3 and 4. [I2-F]

Sometimes the feeling of understanding came only after submitting the assignment when the student had an interview with a TA. Having to explain the code to TA helped some students to understand the parts of the assignment that had been unclear while doing the assignment.

3.2.2.4. Negative Programming Episode → Positive Self-Efficacy Assessment (“− +” -story). The following episodes illustrate rather rare (at least in our data) combinations of a negative programming process followed by positive self-efficacy beliefs. In the context of dealing with a difficulty, in addition to being off-track (not knowing what to do or not being able to solve a problem), students still explicitly expressed that they had confidence that eventually they would be able to proceed and solve the problem (positive self-efficacy assessment). This evaluation was based on either their own abilities (“... *there was never a time when I was just like – was about to break down and cry and say, ‘Oh, my God. I can’t do this. This is way too over my head and difficult.’ It was always a matter of I know the answer is there, so let me just try and find it*”. [I5-J]) or in the context of being in a course (“*I wasn’t really that stressed ‘cause I knew she wouldn’t hand us an impossible assignment*” [I5-J]).

The common feature/intervening condition of both of these episodes is the students’ expectations regarding their own abilities; even though students were having difficulties with solving the current problem they were still confident that eventually they would be able to solve it. Coming up with a solution was just a matter of time. In this scenario, students did not compare their performance with other students or aspects of their previous own programming processes.

As a consequence of a negative episode (dealing with a difficulty) students still reported that they felt negative emotions, such as, annoyance and frustration. However, because of the intervening conditions of self-expectations (expectations of own abilities) and external forces (course-specific), students did not feel as negatively as would have been possible or likely otherwise (“*It just felt like more annoyance and frustration trying to get the code working than absolute confusion*”. [I5-J]). Whether the less negative emotions were a consequence of positive self-efficacy belief or vice versa cannot be deduced from the data.

3.2.3. *Summary of the results*

The four stories highlight different aspects of how and under which conditions CS students build perceptions of their ability as a programmer and a learner. Two of the stories, Negative Programming Episode → Negative Self-Efficacy assessment (“− −” -story) and Positive Programming Episode → Positive Self-Efficacy assessment (“+ +” -story), are perhaps not surprising since it is easy to see how positive outcome (e.g. no compiler errors) leads to positive self-efficacy assessments and vice versa. However, two other stories, Positive Programming Episode → Negative Self-Efficacy assessment (“+ −” -story), and Negative Programming Episode → Positive Self-Efficacy assessment (“− +” -story), provide an interesting variation to the otherwise straightforward line of thinking that positive processes would always lead to positive self-efficacy assessment. On the contrary, the results suggest that students take other aspects into account (consciously or unconsciously) when they construct their self-efficacy perceptions. These aspects include expectations of their fluency with the process, their own abilities, and other students’ abilities. In our discussion, we will focus on the “− +” and “+ −” stories as they may serve as a cue to understand why some students persist regardless of the difficulties they face and others decide to withdraw even if they are doing fine in the course (e.g. get assignments done).

4. Discussion

Reflecting on our narratives describing student experiences with programming assignments, we find two core issues to bring to the attention of the computing education community, and which we believe the community should continue to further explore: previous positive experiences support positive self-efficacy assessment and students’ self-efficacy perception with the process of doing a programming assignment (not just the result). Addressing the first allows us to explore how we might move students from Negative Programming Episode → Negative Self-Efficacy assessment to Negative Programming Episode → Positive Self-Efficacy assessment (“− −” -story → “− +” -story). Consideration of the second may support turning experiences of Positive Programming Episode → Negative Self-Efficacy assessment into Positive Programming Episode → Positive Self-Efficacy assessment (“+ −” -story → “+ +” -story).

4.1. *Previous positive experiences*

Suggestion 1: First programming experiences should be managed to have a much higher percentage of positive experiences than negative ones.

The most exciting result of this work has been to identify that previous “impactful” positive programming experiences led students to think, “even though I’m struggling now, I know I can get there”. Except for the more mundane belief that a professor wouldn’t assign something impossible, it was quite clear that whenever students reflected on previous successes, they could maintain a positive self-efficacy assessment in the context of current struggles.

A number of options exist, which may be able to contribute to this goal. Programming languages *designed* to engage novice programmers such as Alice (Moskal, Lurie, & Cooper, 2004) or Scratch (Resnick et al., 2009) seek to dramatically reduce the student experience with syntactic struggles and to emphasize the excitement or pride induced when a positive result is achieved. Theories of self-efficacy (e.g. Bandura, 1982) and self-regulated learning (e.g. Pintrich & De Groot, 1990) corroborate the pedagogical means to enhance students’ positive self-efficacy perceptions. Additionally, the animation-nature of the programs students produce in these languages strongly encourages incremental development – that is watching to see if the first part of the animation works, then adding something then seeing if that works, etc. Because it is natural to not complete an entire program before “running” the code, students experience many successes for every program they write – especially if specifically guided to do so.

Another option involves starting students with working code and having them manipulate that in some way. Faded worked examples (Caspersen & Bennedsen, 2007) in programming involve breaking down the problem into set design and implementation steps (each scaffolded with more steps). A student is provided with one fully worked example, then given another, very similar example with all but the last step worked out, repeating with more examples reducing the number of provided steps each time. Even though the original worked example is not exactly a “success” of the student themselves, reading through it and understanding it, then getting to check and apply that understanding incrementally through a series of related problems seems likely to meet our goal of increased positive experiences (certainly they involve engagement with significantly more examples than we traditionally see with a “write a program” style assignment). This pedagogical approach is also in line with the theory of the Zone of Proximal Development (Vygotsky, 1978) and may thus be effective. Interestingly, Craig, Chi, and VanLehn (2009) found that even better than simply working with worked examples was having pairs of students working on a problem while watching a video of someone being tutored on a similar problem. It may even be that vicarious success (but one that also shows that struggle is OK) can play a beneficial roll.

4.2. *Perception of the process is important*

Suggestion 2: First programming experiences should seek to give feedback on the entire experience, not just the correctness of their final program.

Instructors often assume programming assignments are a “mastery” experience. We hope (and sometimes believe) that students who successfully complete a programming assignment have mastered the material and process or at least have a positive view of themselves in this regard. But we see from the Positive Programming Episode Negative Self-Efficacy Assessment -stories that this is not the case. Submission of a working program only means the student could produce a working program, not that they always feel they understand the underlying concepts or the process of programming. Not only may students not be able to assess their understanding accurately, but different students may see the “same” situation differently. For reasons related to self-expectations, time required, partner feedback, or comparison to others, they may feel that even if they “got it” they didn’t get it well enough.

This finding would argue for development of structured reflection at the end of assignments (or between them) (Caruso, Hill, VanDeGrift, & Simon, 2011), which might also be structured through peer review (Sitthiworachart & Joy, 2004). Additionally, resources to support students in gaining a more direct and clear feedback of their abilities would likely help. Peer Instruction (e.g. Crouch & Mazur, 2001) is one option by which concept knowledge feedback can be provided quickly and efficiently. However, resources for the more complex assessment of the skill of “programming” seem challenging. Theories of self-regulated learning (e.g. Pintrich & De Groot, 1990) may provide a closer insight to the process of how students construct their self-efficacy beliefs.

One option to affect students’ expectations is to educate students that, in “real” programming, it is normal to have 98% of programming experiences negative. The number of times you “are right” is very limited. Scaffolded work showing videoed tutoring also seems to show potential here – in helping to set a standard regarding the balance of struggle and success in programming. Also important would be techniques to help students recognize and consciously adopt a growth mindset where challenges are seen in a positive light, and are, in fact, instrumental in gaining knowledge and skill (Blackwell, Trzesniewski, & Dweck, 2007).

5. Conclusions and future work

This grounded theory study of students’ perceptions of their experience with programming assignments in a CS1 course seeks to provide a novel, emic account of a mostly unknown phenomenon – what happens to

students when they face programming assignments outside of a structured context. Through a series of interviews, we find that students go through a number of distinct phases in doing a programming assignment and that their primary reflection on the phenomenon is found in the combination of a programming-induced experience (positive or negative) and a subsequent self-efficacy assessment (also positive or negative). While Positive Programming Episode → Positive Self-Efficacy assessment (“++”) and Negative Programming Episode → Negative Self-Efficacy assessment (“--”) experience combinations are common, and likely expected, we identify interesting insights from the Positive Programming Episode → Negative Self-Efficacy assessment (“+-”) and Negative Programming Episode → Positive Self-Efficacy assessment (“-+”) narratives. We suggest that introductory computing instructors consider approaches that reverse the common overwhelming amount of negative feedback in the programming process (e.g. compiler or runtime errors). However, they also need to support reflective processes that help students accurately connect their struggles and successes with programming to their beliefs about whether they “can do” computing.

Ideas for future work born out of this study include placing the results into a larger context by interpreting them in light of self-efficacy and self-regulated learning theories (e.g. Bandura, 1982, Pintrich & De Groot, 1990). Further studies are also needed to shed light on the transition phases (waffling) where students move between positive and negative self-efficacy assessment. Deeper understanding of the factors in play during those phases could guide concrete pedagogical interventions to help students. In addition, the role of emotions as a part of meaningful learning experiences is an overlooked research area in CSE that needs further study. Finally, we also believe the community needs to further explore the impact of pair programming on the experience, the actual abilities of students in contrast to their perception of them, and development of strategies or instruments to provide “unbiased” feedback to students to accurately guide them in assessing their abilities.

Acknowledgements

This work is partly supported by the National Science Foundation under NSF CNS-0933635. We thank the students who so generously talked to us and to Dov Zakis for his help on this project.

Notes

1. Etic accounts look at the phenomenon from the point of view of a person who does not participate in the action/culture that is being studied. Emic accounts look at the phenomenon from the point of view of a person who participates in the action. (see e.g. <http://www.merriam-webster.com/dictionary/emic>)
2. Notation I4-J identifies the interview round (Interview round 4) and the student (J).

References

- Bandura, A. (1982). Self-efficacy mechanism in human agency. *American Psychologist*, 37, 122–147.
- Bandura, A. (1997). *Self-efficacy. The exercise of control*. New York, NY: W.H. Freeman.
- Berglund, A., & Eckerdal, A. (2006). What do CS students strive to learn? *Computer Science Education*, 16, 185–195. doi: 10.1080/08993400600912368.
- Blackwell, L.S., Trzesniewski K.H., & Dweck, C.S. (2007). Implicit theories of intelligence predict achievement across an adolescent transition: A longitudinal study and an intervention. *Child Development*, 78(1), 246–263. doi: 10.1111/j.1467-8624.2007.00995.x.
- Cantwell Wilson, B. (2002). A study of factors promoting success in computer science including gender difference. *Computer Science Education*, 12, 141–164. doi: 10.1076/csed.12.1.141.8211.
- Caspersen, M.E., & Bennedsen, J. (2007). Instructional design of a programming course: A learning theoretic approach. In *Proceedings of the third international workshop on computing education research (ICER '07)* (pp. 111–122). New York, NY: ACM. doi: 10.1145/1288580.1288595.
- Cohen, L., Manion, L., & Morrison, K. (2000). *Research methods in education*. London: Routledge Farmer.
- Craig, S., Chi, M.H.T., & VanLehn, K. (2009). Improving classroom learning by collaboratively observing human tutoring videos while problem solving. *Journal of Educational Psychology*, 101, 779–789.
- Crouch, C., & Mazur, E. (2001). Peer instruction: Ten years of experience and results. *American Journal of Physics*, 69, 970–977. doi: 10.1119/1.1374249.
- Eckerdahl, A., & Thuné, M. (2005). Novice Java programmers' conceptions of “object” and “class”, and variation theory. *SIGCSE Bulletin*, 37, 89–93. doi: 10.1145/1151954.1067473.
- Garvin-Doxas, K., & Barker, L.J. (2004). Communication in computer science classrooms: Understanding defensive climates as a means of creating supportive behaviours. *ACM Journal of Educational Resources in Computing*, 4(1), 1–18. doi: 10.1145/1060071.1060073.
- Glaser, B., & Strauss, A. (1967/1999). *The discovery of grounded theory. Strategies for qualitative research*. New York, NY: Aldine de Gruyter.
- Goldman, K., Gross, P., Heeren, C., Herman, G., Kaczmarczyk, L., Loui, M.C., & Zilles, C. (2008). Identifying important and difficult concepts in introductory computing courses using a delphi process: Selective compression of unicode arrays in java. *SIGCSE Bulletin*, 40(1), 256–260. doi: 10.1145/1352322.1352226.
- Heikkinen, H., Huttunen, R., & Syrjälä, L. (2007). Action research as narrative: Five principles for validation. *Educational Action Research*, 15(1), 5–19. doi: 10.1080/09650790601150709.
- Kinnunen, P., & Simon, B. (2010a). Building theory about computing education phenomena: A discussion of grounded theory. In *Proceedings of the 10th Koli calling international conference on computing education research (Koli Calling '10)*, October 28–31, 2010, Koli, Finland (pp. 37–42). New York, NY: ACM. doi: 10.1145/1930464.1930469.
- Kinnunen, P., & Simon, B. (2010b). Experiencing programming assignments in CS1: The emotional toll. In *Proceedings of the sixth international workshop on computing education research (ICER '10)*, August 9–10, 2010, Aarhus, Denmark (pp. 77–86). New York, NY: ACM. doi: 10.1145/1839594.1839609.
- Kinnunen, P., & Simon, B. (2011). CS majors self-efficacy perceptions in CS1: Results in light of social cognitive theory. In *Proceedings of the seventh international workshop on Computing education research (ICER '11)*, August 9–10, 2011, Provence, USA (pp. 19–26). New York, NY: ACM. doi: 10.1145/2016911.2016917.
- Lincoln, Y., & Guba, E. (1985). *Naturalistic inquiry*. Newbury Park, CA: Sage Publications.

- Lister, R., Adams, E., Fitzgerald, S., Fone, W., Hammer, J., Lindholm, M. . . . Thomas, L. (2004). A multi-national study of reading and tracing skills in novice programmers. *SIGCSE Bulletin*, 35, 119–150. doi: 10.1145/1041624.1041673.
- Moskal, B., Lurie, D., & Cooper, S. (2004). Evaluating the effectiveness of a new instructional approach. *SIGCSE Bulletin*, 36(1), 75–59. doi: 10.1145/1028174.971328.
- Pintrich, P.R., & De Groot, E.V. (1990). Motivational and self-regulated learning components of classroom academic performance. *Journal of Educational Psychology*, 82(1), 33–40. doi: 10.1037/0022-0663.82.1.33.
- Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K. . . . Kafai, Y. (2009). Scratch: Programming for all. *Communications of the ACM*, 52, 60–67.
- Robins, A., Rountree, J., & Rountree, N. (2003). Learning and teaching programming: A review and discussion. *Computer Science Education*, 13, 137–172.
- Schulte, C., & Bennedsen, J. (2006). What do teachers teach in introductory programming? In *Proceedings of the 2006 international workshop on computing education research (ICER '06)*, September 9–10, 2006, Canterbury, UK (pp. 17–28). New York, NY: ACM. doi: 10.1145/1151588.1151593.
- Simon, B., Kinnunen, P., Zaskis, D., & Porter, L. (2010). Experience report: CS1 for majors with media computation. *ACM SIGCSE Bulletin*, 42, 214–218. doi: 10.1145/1822090.1822151.
- Simon, F.S., Robins, A., Baker, B., Box, I., Cutts, Q., . . . Tutty, J. (2006). Predictors of success in a first programming course. In *Proceedings of the 8th Australian conference on computing education (ACE '06) – Volume 52*. January 2006, Hobart, Australia (pp. 189–196). Darlinghurst, Australia: Australian Computer Society, Inc.
- Sitthiworachart, J., & Joy, M. (2004). Effective peer assessment for learning computer programming. In *Proceedings of the 9th annual SIGCSE conference on innovation and technology in computer science education (ITiCSE '04)* (pp. 122–126). New York, NY: ACM. doi: 10.1145/1007996.1008030.
- Sorva, J. (2008). The same but different students' understandings of primitive and object variables. In *Proceedings of the eighth Baltic Sea conference on computing education research (Koli Calling 2008)*. November 13–16, 2008, Koli National Park, Finland (pp. 5–15). New York, NY: ACM. doi: 10.1145/1595356.1595360.
- Strauss, A., & Corbin, J. (1990). *Basics of qualitative research: Grounded theory procedures and techniques*. Newbury Park, CA: Sage Publications.
- Tew, A., Fowler, C., & Guzdial, M. (2005). Tracking an innovation in introductory CS education from a research university to a two-year college. *SIGCSE Bulletin*, 37(1), 416–420. doi: 10.1145/1047124.1047481.
- vanDeGrift, T., Caruso, T., Hill, N., & Simon, B. (2011). Experience report: Getting novice programmers to THINK about improving their software development process. In *Proceedings of the 42nd ACM technical symposium on computer science education* (pp. 493–498). New York, NY: ACM.
- Vygotsky, L.S. (1978). *Mind in society: The development of higher psychological processes*. Cambridge, MA: Harvard University Press.
- Wiedenbeck, S. (2005). Factors affecting the success of non-majors in learning to program. In *Proceedings of the first international workshop on computing education research (ICER '05)*. October 1–2, 2005, Seattle, Washington, USA (pp. 13–24). New York, NY: ACM. doi: 10.1145/1089786.1089788.